ТЕХНИЧЕСКОЕ ЗАДАНИЕ • RUST + BEVY

ОБНОВЛЁН 27.10.2026



ParodiaR
техническое задание

Воксельная RPG-survival на Rust и Bevy: требования, стек, методологии, дерево проекта, архитектура, правила кода и этапы с проверкой «готово, когда».

Требования	**Стек**	**Дерево**	**Правила**	**Этапы**
что и какое	методологии	архитектура	кода и git	0 → 18

Ведёт Claude вместе с Даней • исходник: docs/TZ.md

Содержание

01 Главное за минуту

02 Что строим: видение и границы

03 Требования

04 Стек технологий

05 Методологии: как думаем

06 Дерево проекта

07 Архитектура: кто от кого зависит

08 Шаблон плагина

09 Правила кода

10 Git и документы

11 Этапы

12 Решения и открытые вопросы

13 Словарь

01 Главное за минуту

> Это техническое задание на **ParodiaR** — воксельную RPG-survival на Rust и Bevy. Здесь описано, **что** делаем (требования), **из чего** (стек), **как устроен проект** (дерево папок и правила, кто от кого зависит), **как писать код**, чтобы через полгода в нём не запутаться, и **в каком порядке** строить игру — этапами, у каждого из которых есть проверка «готово».

ЯЗЫК И ДВИЖОК	ПЛАТФОРМА	ЦЕЛЕВОЕ ЖЕЛЕЗО	РЕЖИМ
Rust 1.99 • Bevy 0.20	**Windows 10 • ПК • без интернета**	**Ryzen 5 4650G • встроенная графика**	**одиночная игра, позже кооператив**

Пять правил, на которых держится весь проект:

Правило	Что даёт
Ядро без Bevy. Мир, генерация, меш, физика, сохранение — в отдельном пакете `parodia_core`, который не знает про движок	всё важное проверяется тестами за секунды; при обновлении Bevy ядро не ломается
Одна фича — один плагин — одна папка	понятно, где искать код; плагины не лезут друг к другу во внутренности
Менять мир — только через `World::set_block`	свет, меш, сохранение, сеть узнают об изменении сами
Код появляется вместе с этапом — никаких «заготовок на будущее»	в проекте нет мёртвых файлов, которые путают
Этап закончен, когда выполнена его проверка и сделан коммит	всегда есть рабочая версия, к которой можно вернуться

Как пользоваться документом. Разделы 02–05 — один раз прочитать, чтобы понимать «что и зачем». Раздел 06 (дерево) и 08 (правила кода) — держать открытыми во время работы. Раздел 10 (этапы) — рабочий план: берёшь следующий этап и делаешь его от «Цели» до «Готово, когда». Формулы и алгоритмы к этапам — в «Карте знаний» (`Parodia_Study.pdf`), номера разделов указаны.

02 Что строим: видение и границы

> Большой бесшовный воксельный мир, который генерируется по seed: горы, пещеры, биомы, вода, постройки и вписанные ориентиры. В нём — персонаж с прокачкой, оружием и магией из компонентов, мобы и данжи. Стиль — пиксель-арт в 3D, «красиво как Terraria».

Входит в проект

- Бесконечный по горизонтали мир, по вертикали −3072…+2048 блоков (20 столбиков по 256), без экранов загрузки. Расширение по высоте — смена двух констант.
- Генерация: рельеф, биомы с плавными переходами, пещеры по слоям глубины, руды, деревья, вода, постройки, ориентиры, данжи.
- Строительство: целые блоки, формы (ступеньки, скосы), мини-части (восьмушки, резьба ¼).
- Персонаж: коробка 0,6×1,8×0,6, своя физика по блокам, вид от 1-го/3-го лица.
- Сохранение: seed + только изменения игрока.
- Дальняя прорисовка (как Distant Horizons).
- Позже: мобы, бой, прокачка, магия, инвентарь и крафт.

Не входит (сознательно)

Не делаем	Почему
ММО на сотни игроков, секторы, Redis/NATS	не по силам одному; из старого ТЗ взяли только ядро
Фаза «сначала 2D»	мир сразу 3D
Сглаженный рельеф (marching cubes)	стиль — кубики с формами
Готовые ассеты из магазинов	могут быть недоступны; всё рисуем и генерируем сами
Тяжёлая графика (трассировка лучей, объёмные облака)	встроенная графика не потянет

Откуда взяты решения

Решения, которые уже приняты в Unity-проекте Parodia (размер блока, хитбокс, камера F5, формы блоков, сохранение изменений), **перенесены сюда без изменений** и отмечены в разделе 11. Unity-проект остаётся справочником: если не знаешь, как сделать этап, — посмотри, как он устроен там (`D:\Program\parodia\Parodia\Docs`).

03 Требования

> Требования — это проверяемые утверждения «игра должна…». Функциональные — что игра делает. Нефункциональные — какой она должна быть (скорость, память, удобство). Номера (Ф-01, Н-01) нужны, чтобы ссылаться на них в коммитах и задачах.

Функциональные

№	Требование
Ф-01	Один и тот же seed всегда даёт один и тот же мир, в любом порядке загрузки чанков
Ф-02	Мир подгружается вокруг игрока и выгружается за спиной без остановки игры
Ф-03	Игрок ходит, прыгает, плавает, летает (режим наблюдателя V), не проходит сквозь блоки
Ф-04	Игрок ломает и ставит блоки, формы и мини-части; изменения видны сразу
Ф-05	Мир сохраняется (seed + изменения) и загружается; формат имеет номер версии
Ф-06	Камера: 1-е и 3-е лицо по F5, колесо мыши — прицельный зум
Ф-07	Окно отладки F3: FPS, мс кадра, координаты, чанк, биом, блок под прицелом
Ф-08	Окно графики F10: дальность, дальняя прорисовка, масштаб пикселей, туман
Ф-09	Консоль команд (T или /), каждая система регистрирует свои команды
Ф-10	Дальняя прорисовка рельефа до 2 км
Ф-11	Вода течёт по блокам, у неё есть объём
Ф-12	Постройки, ориентиры и данжи генерируются по seed прямо в мире

Нефункциональные

№	Требование	Как проверить
Н-01	⩾ 60 кадров (⩽ 16,7 мс) на Ryzen 5 4650G, 1080p, масштаб рендера ½, 8 чанков вокруг	окно F3, 1 минута полёта
Н-02	Нет фризов > 50 мс при подгрузке мира	худший кадр в F3 / Tracy
Н-03	Память игры ⩽ 3 ГБ при дальности 8 + дальняя прорисовка	Диспетчер задач
Н-04	Сохранение нетронутого мира — меньше 1 МБ	размер папки сохранения
Н-05	Игра и разработка работают **без интернета**	собрать и запустить с выключенной сетью
Н-06	Весь интерфейс на русском	—
Н-07	**Никакая информация не передаётся только цветом**: статусы, редкость, подсказки — словом, знаком или формой	просмотр интерфейса в оттенках серого

№	Требование	Как проверить
H-08	Ядро (`parodia_core`) собирается и тестируется без Bevy	`cargo test -p parodia_core`
H-09	Первая сборка — один раз; пересборка после правки игрового кода — до ~30 с	замер `cargo run`

04 Стек технологий

> Минимум зависимостей. Каждая сторонняя библиотека — это риск (может не обновиться под новую версию Bevy, может быть недоступна). Поэтому всё, что занимает меньше ~200 строк, пишем сами.

Что	Чем	Зачем	Статус
Язык	Rust stable 1.99, edition 2024	скорость как у C++, ошибки памяти ловит компилятор	установлено
Набор сборки	MSVC + Build Tools 2026 + Windows SDK	линковка под Windows	установлено
Движок	Bevy 0.20	ECS, рендер (wgpu), окно, ввод, звук, ассеты	установлено
Шейдеры	WESL (`.wesl`, расширение WGSL с `import`)	язык шейдеров Bevy 0.20	этап 7
Сериализация	`serde` + `ron` (настройки, данные блоков)	человекочитаемые файлы данных	этап 2
Сохранение	свой бинарный формат (+ `lz4_flex` для сжатия)	маленькие и быстрые сохранения	этап 8
Шум, хэш	**свои** (Perlin/OpenSimplex, SplitMix64)	полный контроль, детерминизм, без зависимостей	этап 4
Отладочный UI	встроенный `bevy_ui` (позже — проверить `bevy_egui` под 0.20)	окна F3/F10, консоль	этап 1
Профилирование	Tracy (функция Bevy `trace_tracy`), RenderDoc	искать, что тормозит	с этапа 5
Качество кода	`rustfmt`, `clippy`	единый стиль, подсказки об ошибках	этап 0
Контроль версий	git (из консоли) + GitHub (приватный)	история и откат	этап 0
Редактор	VS Code + rust-analyzer	подсказки, переход к определению	установлено

Важно: версии Bevy. Русский справочник по Bevy 0.20 — `docs/Bevy_RU.pdf`. Bevy выходит примерно раз в 3–4 месяца, и каждая версия ломает часть API. Правило: **версию меняем только сознательно**, отдельным этапом, с руководством по миграции (Migration Guide) на сайте Bevy. В `Cargo.toml` пишем `bevy = "0.20"` — это значит «любая 0.20.x», на 0.21 Cargo сам не перейдёт. Документация ровно твоей версии — `cargo doc --open` (работает без интернета).

05 Методологии: как думаем

> Методология — это набор привычек, которые не дают проекту превратиться в «говнокод». Ниже — те, что реально нужны одному разработчику игры. Каждая объяснена одной фразой и примером.

1. Функциональное ядро, императивная оболочка

Вся логика, которую можно описать как «на входе данные — на выходе данные», живёт в ядре и не знает про окно, кадры и Bevy. Bevy — тонкая «оболочка»: берёт результат ядра и показывает его.

- **Пример:** мешер принимает чанк и соседей, возвращает `Vec<u32>` упакованных вершин. Он ничего не знает про `Mesh`, `Handle`, `Commands`. Плагин рендера превращает этот массив в меш Bevy.
- **Почему:** такую функцию можно проверить тестом (`assert_eq!(faces, 6)`) без запуска игры. В Unity-проекте это было `Game.World.Data` — здесь то же, но компилятор сам запрещает ядру импортировать Bevy.

2. Данные отдельно от поведения (ECS)

Bevy построен на ECS (Entity Component System):

Понятие	Что это	Пример
Entity (сущность)	просто номер	игрок, моб, чанк на экране
Component (компонент)	данные, прикреплённые к сущности	`Transform`, `Health`, `ChunkPos`
System (система)	функция, которая обрабатывает все сущности с нужными компонентами	«всем, у кого есть `Velocity` и `Transform`, сдвинуть позицию»
Resource (ресурс)	одна общая вещь на всю игру	`VoxelWorld`, `GraphicsOptions`
Message / Event	сообщение «что-то случилось»	`BlockChanged`, `PlayerDied`

- **Правило:** компонент — только данные (без методов с логикой игры). Логика — в системах.
- **Главное правило воксельной игры: блок — не сущность.** Блоки лежат в массиве чанка внутри ресурса `VoxelWorld`. Сущностью является только чанк **на экране** (его меш). 4 миллиарда сущностей-блоков убьют любую игру.

3. Одна точка изменения и события

Мир меняется только через `World::set_block`. Функция сама: помечает чанк «грязным» (нужно перестроить меш), запоминает изменение для сохранения, отправляет сообщение `BlockChanged`. Свет, вода, обрушение, звук подписываются на это сообщение — и никто не меняет блоки «в обход».

4. Данные вместо кода (data-driven)

Блоки, биомы, предметы, правила генерации — это **файлы данных** (`assets/data/*.ron`), а не `match` на 500 строк. Добавить новый блок = дописать запись в файл, без перекомпиляции логики.

```
// assets/data/blocks.ron
[
  (name: "stone", title: "Камень", solid: true, transparent: false,
   hardness: 1.5, textures: (all: "stone")),
  (name: "grass", title: "Трава", solid: true, transparent: false,
   hardness: 0.6, textures: (top: "grass_top", side: "grass_side", bottom: "dirt")),
]
```

5. YAGNI и вертикальный срез

YAGNI («You Aren't Gonna Need It» — «тебе это не понадобится»): не пишем код «на будущее». Вертикальный срез: каждый этап доводит **маленький кусок** игры до рабочего состояния через все слои (данные → логика → картинка → проверка), а не делает «весь рендер», потом «всю логику».

6. Детерминизм

Генерация зависит только от seed и координат. Никакого `rand::random()`, времени, порядка загрузки, `HashMap` с перебором в генерации. Проверяется тестом: «сгенерировать чанк дважды в разном порядке — хэш одинаковый».

7. Сначала замер, потом оптимизация

Не оптимизировать «на глаз». Сначала окно F3 / Tracy показывает, что тормозит, потом чиним именно это и снова меряем. Каждая оптимизация записывается в журнал с цифрами «было / стало» (в миллисекундах, не в FPS).

8. Решения записываются (ADR)

ADR (Architecture Decision Record — запись архитектурного решения): каждое важное решение — одна запись в `docs/DECISIONS.md`: «дата — решение — почему — какие были варианты». Через полгода ты не будешь помнить, почему чанк 32, а не 16, — а файл будет.

06 Дерево проекта

> Проект — это **рабочее пространство Cargo** (workspace): несколько пакетов в одной папке, которые собираются вместе. Сначала пакетов два: ядро и игра. Ниже — целевое дерево. Папки создаются, **когда до них доходит этап**, а не заранее; номер этапа указан справа.

Сейчас (этапы 0–1) проект проще — один пакет:

```
rust/                    C:\Users\xcx7276\Desktop\rustssd\rust
├── Cargo.toml  Cargo.lock  .gitignore  CLAUDE.md
├── .cargo/config.toml   быстрый линковщик
├── assets/fonts/        шрифт с кириллицей
├── docs/                Т3, справочник Bevy, START.md, DECISIONS.md
└── src/
    ├── main.rs          только сборка плагинов
    ├── camera.rs        летающая камера
    ├── debug.rs         окно F3
    └── test_scene.rs    пол, кубы, солнце (временно)
```

Дерево ниже — **целевое**: к нему переходим на этапе 2.

```
ParodiaR/                          C:\Users\xcx7276\Desktop\rustssd\rust
├── Cargo.toml                     рабочее пространство + профили сборки        этап 0
├── Cargo.lock                     точные версии зависимостей (в git!)
├── .gitignore                     /target и временные файлы                     этап 0
├── .cargo/config.toml             быстрый линкер (по желанию)                   этап 0
├── rustfmt.toml                   стиль форматирования                          этап 0
├── README.md                      описание для GitHub (1 экран)                 этап 0
├── CLAUDE.md                      память ИИ-помощника о проекте                есть
│
├── docs/                          документация
│   ├── TZ.md  ParodiaR_TZ.pdf     это ТЗ                                        есть
│   ├── ARCHITECTURE.md            как устроено сейчас (обновлять с кодом)       этап 0
│   ├── DECISIONS.md               журнал решений (ADR)                          есть
│   ├── JOURNAL.md                 что сделано по датам, замеры                  этап 0
│   └── build/                     скрипты сборки PDF                            есть
│
├── assets/                        всё, что грузит игра во время работы
│   ├── textures/blocks/           плитки блоков 16×16 (.png)                    этап 3
│   ├── textures/items/  ui/       предметы, интерфейс                           этап 13+
│   ├── shaders/                   blocks.wesl, water.wesl, sky.wesl             этап 7
│   ├── data/                      blocks.ron, biomes.ron, items.ron             этап 2
│   ├── structures/                шаблоны построек                              этап 15
│   ├── fonts/                     шрифт с кириллицей                            этап 1
│   └── sounds/                                                                  позже
│
├── crates/
│   ├── parodia_core/              ЯДРО: без Bevy, только Rust                   этап 2
│   │   ├── Cargo.toml
│   │   ├── src/
│   │   │   ├── lib.rs             список модулей ядра
│   │   │   ├── math/              int3, dir, aabb, hash, координаты чанков     этап 2
│   │   │   ├── block/             BlockId, реестр свойств, формы, мини-части    этап 2, 9
│   │   │   ├── chunk/             секция 32³, столбик ×8, палитра               этап 2
│   │   │   ├── world/             VoxelWorld, set_block, изменения, подгрузка   этап 2, 5
│   │   │   ├── gen/               шум, рельеф, биомы, пещеры, руды, деревья     этап 4, 11
│   │   │   │   ├── structures/    шаблоны, пазл-постройки                       этап 15
│   │   │   │   ├── landmarks/     SDF-ориентиры                                 этап 16
│   │   │   │   └── dungeons/      граф → комнаты → коридоры                     этап 17
│   │   │   ├── mesh/              грани, жадный меш, AO, упаковка вершин        этап 3, 7
│   │   │   ├── light/             заливка света                                 этап 10
│   │   │   ├── fluid/             вода                                          этап 14
│   │   │   ├── physics/           коробка тела, луч DDA                         этап 6
│   │   │   ├── lod/               дальние плитки                                этап 12
│   │   │   └── save/              формат файла, регионы                         этап 8
│   │   └── tests/                 большие тесты (детерминизм, сохранение)       этап 2
│   │
│   └── parodia_game/              ИГРА: Bevy-приложение                         этап 1
│       ├── Cargo.toml
│       └── src/
│           ├── main.rs            только App::new() + список плагинов
│           ├── state.rs           AppState: Загрузка, Меню, Игра, Пауза         этап 8
│           ├── sets.rs            порядок групп систем (SystemSet)              этап 5
│           ├── settings/          GraphicsOptions, клавиши (файл настроек)      этап 1
│           ├── debug/             F3, F10, консоль команд                       этап 1
│           ├── world/             ресурс мира, подгрузка, задачи в фоне         этап 2, 5
```

```
│           ├── render/          меш чанков, материал блоков, массив текстур   этап 3, 7
│           ├── player/          ввод, движение, камера, ломать/ставить        этап 1, 6
│           ├── ui/              HUD, инвентарь, меню                          этап 8, 13
│           ├── save/            меню миров, автосохранение                    этап 8
│           ├── water/  sky/  lod/                                             этапы 12–14
│           └── mobs/                                                          этап 18
│
└── tools/                      вспомогательные программы (позже)
    └── gen_preview/            картинка карты мира по seed — без игры        этап 11
```

Почему два пакета, а не один

Если всё в одном пакете	Если ядро отдельно
можно случайно вызвать Bevy из генерации — и ядро привязано к движку навсегда	компилятор **не даст**: в `parodia_core/Cargo.toml` нет Bevy
тесты собирают весь Bevy (минуты)	`cargo test -p parodia_core` — секунды
обновление Bevy ломает всё	обновление Bevy трогает только `parodia_game`
позже сервер для кооператива придётся выпиливать	сервер = ещё один пакет, использующий то же ядро

Корневой Cargo.toml

```toml
[workspace]
resolver = "3"
members = ["crates/*"]
default-members = ["crates/parodia_game"]   # cargo run запускает игру

[workspace.package]
edition = "2024"
version = "0.1.0"

[workspace.dependencies]          # версии в одном месте для всех пакетов
bevy = "0.20"
serde = { version = "1", features = ["derive"] }
ron = "0.12"

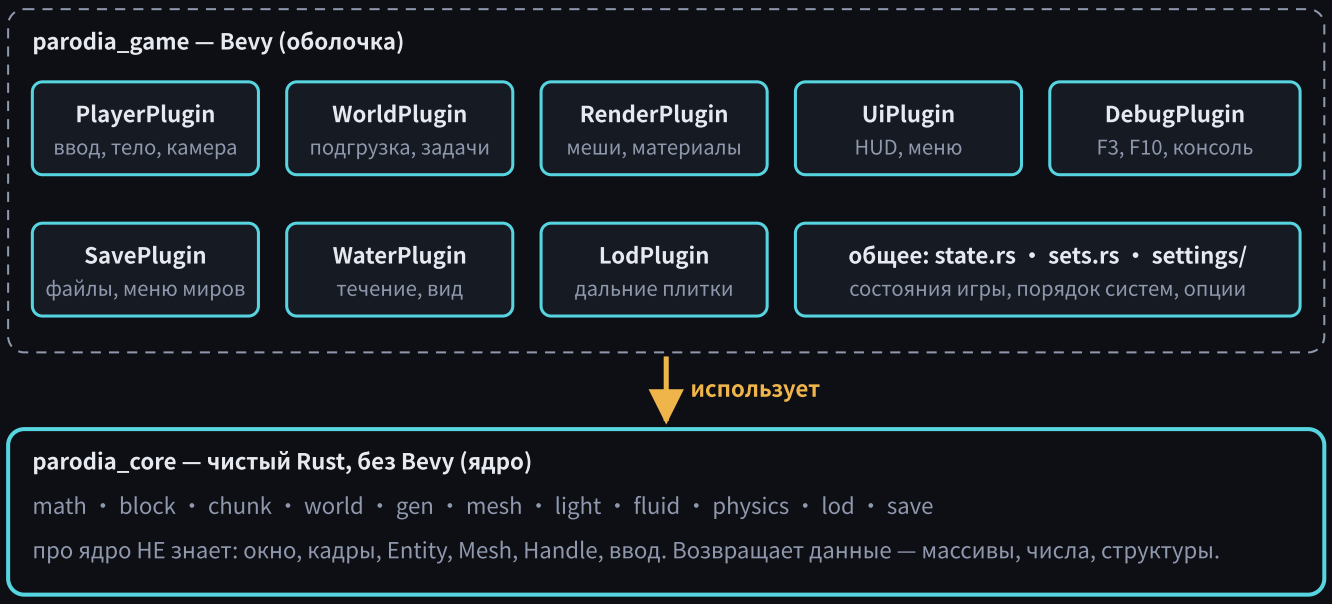
[profile.dev]
opt-level = 1                     # свой код — быстро собирается

[profile.dev.package."*"]
opt-level = 3                     # Bevy и библиотеки — быстро работают

[profile.release]
lto = "thin"
codegen-units = 1
```

Проверь номер версии `ron` перед добавлением: `cargo add ron -p parodia_core` сам поставит актуальную. Версии в примерах — ориентир, а не закон.

07 Архитектура: кто от кого зависит



Правила зависимостей

1. `parodia_core` не зависит ни от чего, кроме мелких библиотек (`serde`). **Bevy в нём запрещён.**
2. Внутри ядра — снизу вверх: `math → block → chunk → world → (gen, mesh, light, fluid, physics, lod, save)`. Нижние модули не импортируют верхние.
3. Плагины игры **не трогают внутренности друг друга**. Общаются через:
4. ресурсы ядра (`VoxelWorld` — все читают, меняют через `set_block`);
5. сообщения (`BlockChanged`, `ChunkMeshed`, `PlayerTeleported`);
6. публичные компоненты (`Player`, `ChunkPos`).
7. `main.rs` — только сборка приложения. Логики в нём нет.

Путь одного действия: игрок ломает блок

Шаг	Где	Что происходит
1	`player/interaction.rs`	ЛКМ → луч DDA из `parodia_core::physics::raycast` → блок (x, y, z)
2	`player/interaction.rs`	`world.set_block(pos, AIR)`
3	`parodia_core::world`	чанк помечен «грязным», изменение записано в список для сохранения, в очередь — `BlockChanged`
4	`world/`	система переносит сообщения ядра в сообщения Bevy
5	`render/chunk_mesh.rs`	видит грязный чанк → задача в фоне: `parodia_core::mesh::build(chunk, соседи)`
6	`render/chunk_mesh.rs`	задача готова → массив вершин → `Mesh` Bevy → подменяет меш у сущности чанка

Шаг	Где	Что происходит
7	`light/`, `water/`, `save/`	реагируют на `BlockChanged` каждый по-своему

Порядок систем в кадре

Порядок задаётся группами (SystemSet) в `sets.rs`, а не «как получилось»:

```
Input → Player → WorldEdit → Simulation → Streaming → Meshing → Camera → Ui
```

Физика тела, вода и мобы — в `FixedUpdate` (постоянный шаг, например 30 раз в секунду; по умолчанию у Bevy 64), картинка и интерфейс — в `Update`.

08 Шаблон плагина

> Все плагины устроены одинаково. Открыл любую папку — сразу знаешь, где что лежит.

```
crates/parodia_game/src/player/
├── mod.rs            pub struct PlayerPlugin + регистрация всего ниже
├── components.rs     Player, Velocity, OnGround — только данные
├── resources.rs      PlayerSettings — общие настройки плагина
├── messages.rs       PlayerJumped, PlayerLanded — что плагин сообщает другим
├── movement.rs       системы движения (одна задача — один файл)
├── camera.rs         системы камеры
└── interaction.rs    ломать/ставить
```

```rust
// crates/parodia_game/src/player/mod.rs
use bevy::prelude::*;

mod camera;
mod components;
mod interaction;
mod movement;

pub use components::Player; // наружу — только то, что нужно другим

/// Игрок: ввод, движение коробки по блокам, камера, ломать/ставить.
pub struct PlayerPlugin;

impl Plugin for PlayerPlugin {
    fn build(&self, app: &mut App) {
        app.add_systems(Startup, movement::spawn_player)
            .add_systems(FixedUpdate, movement::move_player)
            .add_systems(Update, (camera::follow_player, interaction::break_and_place));
    }
}
```

```rust
// crates/parodia_game/src/main.rs — всё приложение на один экран
use bevy::prelude::*;

mod debug; mod player; mod render; mod settings; mod world;

fn main() {
    App::new()
        .add_plugins(DefaultPlugins)
        .add_plugins((
            settings::SettingsPlugin,
            world::WorldPlugin,
            render::RenderPlugin,
            player::PlayerPlugin,
            debug::DebugPlugin,
        ))
        .run();
}
```

Про сообщения в Bevy 0.20. В новых версиях Bevy буферные события называются **Message** (`add_message`, `MessageWriter`, `MessageReader`), а слово Event оставлено для «наблюдателей» (observers). Точные имена смотри в `cargo doc --open` → `bevy::ecs::message`. Если в примерах из интернета видишь `EventReader` — это старая версия.

09 Правила кода

> Эти правила и есть защита от «говнокода». Нарушать можно, но осознанно и с комментарием «почему».

Имена (так принято во всём Rust)

Что	Стиль	Пример
Файлы, модули, функции, переменные	`snake_case`	`chunk_mesh.rs`, `fn build_mesh()`
Типы, трейты, плагины, перечисления	`CamelCase`	`VoxelWorld`, `PlayerPlugin`, `BlockId`
Константы	`SCREAMING_SNAKE_CASE`	`CHUNK_SIZE`, `PLAYER_HEIGHT`
Плагин	`<Что>Plugin`	`WaterPlugin`
Сообщение	что случилось, в прошедшем времени	`BlockChanged`, `ChunkLoaded`
Система	глагол + что	`move_player`, `spawn_chunk_meshes`

Имена — английские (так принято в коде и так понятнее ошибки компилятора), **комментарии и документация — по-русски**.

Размеры

Что	Ориентир	Если больше
Функция	до ~50 строк	разбить на шаги с понятными именами
Файл	до ~400 строк	разбить по задачам
Система	одна задача	две задачи — две системы
Параметры системы	до ~5–6	вынести часть в `SystemParam` или разделить систему
Вложенность	до 3 уровней `if/for`	ранний `return`, `continue`, вынести функцию

Ошибки

- В ядре функции, которые могут не получиться, возвращают `Result<T, Ошибка>` или `Option<T>`.
- В игре — `warn!` / `error!` в лог и продолжаем работу. Игра не падает из-за битого файла настроек.
- `unwrap()` и `expect()` — только там, где ошибка = баг программиста, и с объяснением: `expect("реестр блоков загружен на старте")`.

- `panic!` в горячем цикле — никогда.

Числа и константы

- «Магических чисел» нет: `16`, `0.55`, `1.8` — в константах ядра (`math::consts`) или в настройках (`assets/data/*.ron`).
- Единицы: **1.0 = 1 метр = 1 блок**. Время — в секундах (`f32`). Углы — в радианах внутри, в градусах в настройках.
- Оси (как в Bevy): **Y вверх**, правая система координат, «вперёд» камеры = −Z.

Шейдеры и шум

- В буферах шейдера (storage, uniform) **нет** `vec3` — он выравнивается по 16 байт. Только `u32`, `f32`, `vec2`, `vec4` или `vec3` + поле-заполнитель.
- В шум — только координаты с масштабом (`x / масштаб`): в целых узлах шум Перлина равен 0.
- Общие функции шейдеров (свет, туман, облака) — в отдельных файлах и подключаются через `import` (WESL, без `#`), а не копируются.

Производительность (всегда, а не «потом»)

- Блок — не сущность. Чанк на экране — сущность.
- В горячих циклах (меш, генерация) — без выделения памяти: буферы создаются один раз и переиспользуются.
- Тяжёлое (генерация, меш, свет больших областей) — в фоне (`AsyncComputeTaskPool`), главный поток только забирает готовое.
- Лимиты работ на кадр (сколько чанков сгенерировать, сколько мешей отправить).

Документация в коде

```rust
/// Ставит блок и сообщает об изменении всем подписчикам.
/// Это единственный способ изменить мир (см. ТЗ, раздел 05, правило 3).
///
/// Возвращает `Err`, если чанк не загружен — незагруженное не трогаем.
pub fn set_block(&mut self, pos: IVec3, id: BlockId) -> Result<(), WorldError> {
```

`///` — описание для `cargo doc` (видно при наведении в VS Code). Комментарий пишет **почему**, а не **что**: «что» видно из кода.

Тесты

Что тестировать	Пример
Границы и отрицательные координаты	блок (−1, 0, −1) лежит в чанке (−1, 0, −1), индекс (15, 0, 15)

Что тестировать	Пример
Записал — прочитал	`set_block` → `get_block`
Детерминизм	один seed, два порядка генерации → одинаковый хэш чанков
Сохранение	записать → прочитать → мир совпадает; старая версия формата читается
Меш	один блок в пустоте → 6 граней; два рядом → 10
Физика	коробка не проходит в стену; автоподъём 0,55 работает, 1 блок — нет

Маленькие тесты — внизу того же файла в `#[cfg(test)] mod tests`, большие — в `crates/parodia_core/tests/`.

Антипаттерны: чего не делать

Антипаттерн	Почему плохо	Как правильно
«Бог-плагин» на 3000 строк	невозможно найти и поменять	плагин на фичу, файл на задачу
Каждый блок — Entity	миллионы сущностей, игра встаёт	блоки в массиве чанка
Логика мира в системах рендера	нельзя тестировать, ломается при обновлении Bevy	логика в ядре
Менять блоки напрямую в массиве чанка	свет, меш, сохранение не узнают	только `set_block`
`Query` на 8 компонентов с `&mut` всего	система делает всё; конфликты при параллельности	разделить систему
Строки как идентификаторы в горячем коде	медленно, опечатки	`BlockId(u16)`, строки только в файлах данных
Копипаста «как в соседнем файле»	исправишь баг в одном месте, в другом останется	общая функция в ядре
Закомментированный старый код	мусор, путает	удалить — он есть в git
«Сделаю универсально на будущее»	80 % «будущего» не наступает	YAGNI, делать нужное этапу

Перед каждым коммитом

```
cd C:\Users\xcx7276\Desktop\rustssd\rust
cargo fmt
cargo clippy --all-targets -- -D warnings
cargo test -p parodia_core
cargo run
```

Если `clippy` ругается — исправить, а не заглушать. Каждое предупреждение clippy — объяснение на русском и ссылка на правило: это бесплатный учитель Rust.

10 Git и документы

Ветки и коммиты

- `main` — всегда собирается и запускается.
- На каждый этап — ветка: `stage-03-chunk-mesh`. Этап готов → слить в `main` → метка `v0.3`.
- Сообщение коммита: **что сделано**, по-русски, с номером этапа или требования:

```
этап 3: меш одного чанка (только видимые грани), тест на 6 граней
```

- Коммитить часто: каждый рабочий шаг. Маленький коммит легко откатить.
- `Cargo.lock` — **в git** (это игра, а не библиотека: точные версии важны). `/target` — **не в git**.

Какой документ когда обновлять

Файл	Для кого	Когда обновлять
`README.md`	работодатель / гость GitHub	при заметной новой возможности
`CLAUDE.md`	ИИ-помощник (и ты)	в конце каждой рабочей сессии: статус, что дальше
`docs/TZ.md`	ты	меняются требования или этапы
`docs/ARCHITECTURE.md`	ты	меняется структура: новый плагин, модуль, правило
`docs/DECISIONS.md`	ты через полгода	принято важное решение (+ почему и варианты)
`docs/JOURNAL.md`	ты	конец этапа: что сделано, замеры «было / стало»

11 Этапы

> Каждый этап — законченный кусок, который можно запустить и проверить. Не переходить к следующему, пока не выполнено «Готово, когда». В скобках — разделы «Карты знаний» (`Parodia_Study.pdf`), где лежат формулы.

0 • Порядок в проекте — 1 вечер

Цель: один пакет, код разложен по файлам-модулям, быстрая сборка, git.

Сделать: `src/main.rs` только собирает плагины; каждая тема — свой файл (`camera.rs` , `debug.rs` , `test_scene.rs`); `.cargo/config.toml` с `rust-lld.exe` ; папки проекта в исключениях Защитника Windows; первый коммит.

Учить: модули Rust (`mod` , `pub` , `use`), `cargo run` , `cargo fmt` , базовый git. Подсказка по шагам — `docs/START.md` .

◆ **Готово, когда:** `cargo run` **открывает окно, пересборка после правки своего кода — меньше минуты, коммит сделан.**

Изменение 27.10: рабочее пространство из двух пакетов (`parodia_core` + `parodia_game`) переносится на этап 2 — когда появится код ядра. До этого — один пакет: проще для старта без опыта в Rust.

1 • Окно, камера, отладка — 1 неделя

Цель: 3D-сцена: свободный полёт камеры (WASD + мышь), один куб, окно F3 с FPS и миллисекундами кадра, шрифт с кириллицей.

Статус 27.10: файлы с подробными комментариями уже в проекте — `src/camera.rs` , `src/debug.rs` , `src/test_scene.rs` . Задача — прочитать, запустить, поменять числа (скорость, чувствительность, цвета) и понять, что на что влияет.

Сделать: плагины `settings` (опции из файла), `debug` (F3), `player` (пока только летающая камера).

Учить: ECS: Entity, Component, System, Resource, Query; `Transform` ; ввод с клавиатуры и мыши; захват курсора. Готовый пример камеры — справочник Bevy, раздел 07; окно F3 — раздел 12 (встроенный шрифт без кириллицы — нужен свой `.ttf`).

◆ **Готово, когда: летаю вокруг куба, F3 показывает FPS, мс и координаты на русском.**

2 • Ядро мира и тесты — 1–2 недели

Цель: разделить проект на два пакета (раздел 06: `parodia_core` + `parodia_game`); в `parodia_core` : координаты, `BlockId` , `Chunk` , `VoxelWorld` с `get_block` / `set_block` , реестр блоков из `blocks.ron` .

Учить: владение и заимствование, `struct` , `impl` , `HashMap` , `div_euclid` , `#[test]` , `serde` + `ron` . (Карта: 03)

◆ **Готово, когда: 10+ тестов зелёные, включая отрицательные координаты и границы чанков; в игре ничего не поменялось — это нормально.**

3 • Меш одного чанка — 1–2 недели

Цель: мешер в ядре (только видимые грани) → `Mesh` Bevy → чанк с текстурами блоков на экране.
Учить: вершины, индексы, нормали, UV; `Mesh` и `StandardMaterial` в Bevy; текстуры без размытия (nearest). (Карта: 09, шаги 1)

◆ **Готово, когда: чанк из разных блоков виден с текстурами; тест «один блок = 6 граней, два рядом = 10» проходит.**

4 • Рельеф из seed — 1–2 недели

Цель: свой шум (Perlin + fBm) и хэш в ядре, генератор: высота → слои (трава, земля, камень). Поле 8×8 чанков.
Учить: шум, октавы, хэш, трейты (`trait WorldGenerator`). (Карта: 03, 04)
Сразу так: 2D-шум — один раз на столбец (кэш); координаты в шум — только с масштабом (`x / 110.0`): в целых узлах Перлин всегда 0.

◆ **Готово, когда: холмы на экране; тест «тот же seed — тот же хэш поля, в любом порядке генерации».**

5 • Мир без края — 2 недели

Цель: подгрузка столбиков 32 × 256 × 32 (секции 32^3 внутри — только нужные по высоте) вокруг камеры, выгрузка за спиной; генерация и меш — в фоновых задачах; лимиты за кадр; `sets.rs` с порядком систем.
Учить: `AsyncComputeTaskPool` , `Send` , `Arc` , SystemSet, Tracy. (Карта: 05, 20)

◆ **Готово, когда: лечу 2 минуты в одну сторону — мир не кончается, худший кадр < 50 мс (H-02), память не растёт бесконечно.**

6 • Персонаж в мире — 2 недели

Цель: коробка 0,6×1,8×0,6, движение по осям Y→X→Z, автоподъём 0,55, прыжок 1,25; луч DDA; ЛКМ ломать, ПКМ ставить; камера F5, зум колесом; полёт V.
Учить: `FixedUpdate` и интерполяция, AABB, DDA. (Карта: 18, 19) Справка: `Parodia_Motion.pdf` из Unity-проекта.

◆ **Готово, когда: чек-лист: прохожу в проход 1×2, не лезу на 1 блок без прыжка, не застреваю на углах, сломанный блок исчезает сразу.**

7 • Быстрый меш — 2–3 недели

Цель: жадный (потом бинарный) меш, упаковка вершины в 8 байт → затем «одна грань = один u32» (инстансинг квадрата, 6 частей меша по направлениям), сортировка чанков от ближнего к дальнему, свой материал и шейдер WESL, массив текстур, AO.
Помнить: в буферах шейдера нет `vec3` (выравнивание 16 байт); мерить на полном мире, а не на одном чанке.
Учить: побитовые операции, свой `Material` и атрибуты вершин в Bevy, WESL. (Карта: 09; справочник Bevy: 09)

◆ **Готово, когда: замер «было / стало» в журнале: число треугольников и мс кадра; картинка не хуже.**

8 • Сохранение и меню — 1–2 недели

Цель: формат «seed + изменения» с подписью и версией, регион-файлы, сжатие; состояния игры (меню → загрузка → игра → пауза); меню миров.

Учить: `std::fs`, `Read` / `Write`, `States` в Bevy, `bevy_ui`. (Карта: 05)

◆ **Готово, когда: построил, вышел, зашёл — всё на месте; нетронутый мир < 1 МБ (H-04); тест «старая версия формата читается».**

9 • Формы и мини-части — 2 недели

Цель: перенести из Unity: полублоки, ступеньки, скосы, восьмушки, резьба ¼ — номера блоков, меш, коробки столкновений, прицел по под-сетке.

Учить: битовые маски (`u8`, `u64`), повороты в сетке. Справка: `Parodia_Architecture.pdf`.

◆ **Готово, когда: ставлю и ломаю все формы; тесты коробок и масок проходят.**

10 • Отсечение и свет — 2–3 недели

Цель: отсечение по сторонам граней и пещерное отсечение; свет неба и блоков заливкой 0…15, день/ночь в шейдере; свет по стороне грани; **цвет по затенению** (3 цвета на блок, градиент по AO); свой упрощённый расчёт света вместо PBR (PBR размывает AO); туман в цвет неба.

Учить: BFS, очереди, видимость. (Карта: 10, 12)

◆ **Готово, когда: в пещере мс кадра заметно меньше, чем без отсечения (замер в журнале); факел освещает пещеру, ночь темнеет без пересчёта.**

11 • Живой мир: биомы, пещеры, деревья — 3–4 недели

Цель: климат и биомы с плавными переходами, хребты и искажение домена, 3D-плотность (обрывы, нависания), пещеры по слоям глубины, руды, деревья; программа-превью карты в `tools/`.

Учить: ridged-шум, domain warping, смешивание биомов; прореживание 3D-шума (узлы через 4), границы-экстремумы. (Карта: 04, 05)

◆ **Готово, когда: превью показывает карту по seed без запуска игры; на границе биомов нет «стены».**

12 • Дальняя прорисовка — 3 недели

Цель: плитки LOD (квадродерево/кольца), юбки, выбор уровня по ошибке в пикселях, построение в фоне, окно F10.

Учить: LOD, clipmap. (Карта: 11)

◆ **Готово, когда: видно горы за 2 км, H-01 выполняется.**

13 • Инвентарь и интерфейс — 2 недели

Цель: хотбар, окно инвентаря, иконки блоков, консоль команд; всё подписано словами и знаками (H-07).

◆ **Готово, когда: весь интерфейс читается в оттенках серого.**

14 • Вода — 3 недели

Цель: течение с объёмом (только активные клетки), плавание, шейдер: Френель, глубина (Бер–Ламберт), пена у берега.

Учить: клеточные автоматы, буфер глубины в шейдере. (Карта: 13)

◆ **Готово, когда: вылил воду в яму — заполнилась ровно на свой объём; у берега прозрачно, вдали темно.**

14.1 • Трава, ветер, облака — 2 недели

Цель: трава-билборды инстансингом, пятна и акценты, ветер из двух шумов, «рисованная» анимация 5–8 кадров/с со сдвигом фазы, отгибание от игрока и мобов, мягкий тун-свет, тени облаков. (Карта: 16)

◆ **Готово, когда: поле травы шевелится, расступается вокруг игрока и не съедает больше ~1 мс кадра (замер в журнале).**

15 • Постройки — 3 недели

Цель: шаблоны с поворотом, фундамент, сетка регионов для расстановки, подгонка рельефа вокруг; затем пазл-постройки (разъёмы).

◆ **Готово, когда: по миру стоят дома, не висят в воздухе и не утоплены в холм.**

16 • Вписанные ориентиры — 3 недели

Цель: реестр ориентиров по регионам, оценка места, SDF-хребет по кривой, SDF-пещера по графу, смешивание с плотностью. (Карта: 07)

◆ **Готово, когда: «любимый хребет» и «пещера-приключение» встречаются в мире без видимого шва, видны и вблизи, и в дальней прорисовке.**

17 • Данжи — 3–4 недели

Цель: граф-сценарий с ключами и замками → раскладка комнат → коридоры A* в 3D → проверка проходимости. (Карта: 08)

◆ **Готово, когда: 100 сгенерированных данжей проходят автоматическую проверку «все комнаты достижимы, ключ раньше замка».**

18+ • Мобы, бой, прокачка, магия — дальше

Цель: модели из частей на шарнирах, суставы и рэгдоллы (ограничения C, J, λ — Карта: 18), поиск пути, дальность симуляции; оружие как данные; заклинания из компонентов (форма + эффект + модификатор). Расписать подробно, когда дойдём до этапа 17.

12 Решения и открытые вопросы

Приняты (перенесены из Parodia)

Что	Значение
Масштаб	1 единица = 1 м = 1 блок
Высота мира	было −3000…+2000; в ParodiaR округлено до −3072…+2048 (см. ниже)
Хитбокс игрока	коробка 0,6 × 1,8 × 0,6 м; автоподъём 0,55; прыжок 1,25; склон 45°
Камера	1-е лицо по умолчанию, F5 — 3-е; колесо — прицельный зум, не дальность
Формы блоков	минимальный фиксированный набор: слои, ступеньки, скосы, углы; мини-части — восьмушки и резьба ¼
Изменение мира	только `set_block`
Сохранение	seed + изменения игрока; подпись и номер версии формата
Мир	один бесшовный; данжи, шахты, деревни — структуры по seed прямо в мире
Сеть	сначала одиночная игра; ядро пишется так, чтобы позже был кооператив 2–8 с хостом
Стиль	пиксель-арт 16 пикс/м, контур 1 px, свет сверху-слева, ⩽ 4 ступени на материал
Доступность	никакой информации только цветом (H-07)
Обрушение	BFS только при разрушении, лимит за кадр, якорь — коренная порода

Новые для ParodiaR

Что	Значение	Почему
Движок	Bevy 0.20, Rust	полный контроль, без магазинов ассетов
Структура	workspace: `parodia_core` (без Bevy) + `parodia_game`	раздел 06
Блок в памяти	`BlockId(u16)` + палитра в чанке	память и диск
Шаг симуляции	`FixedUpdate`, 30 раз в секунду	одинаково на любом FPS
Чанк (вариант А)	данные и меш — **секция 32^3**; 8 секций друг над другом = **столбик 32 × 256 × 32** — единица подгрузки, сохранения и карты высот	выгоды высокого чанка (карта высот и биом на столбик, кэш 2D-шума ×256, меньше позиций для подгрузки) без минусов: правка блока перестраивает меш только одной секции 32^3; бинарный меш на `u32` (как у Tantan)
Высота мира	−3072…+2048 = 20 столбиков = 160 секций; константы в `parodia_core::math::consts`	кратно 256 — границы мира совпадают со столбиками; память от высоты не зависит (в ОЗУ только чанки рядом)

Что	Значение	Почему
Блоки в памяти на старте	массив + палитра в секции; RLE — позже для нетронутых и дальних столбиков	много правок мира (мини-части, вода) — массив проще и быстрее для случайного доступа

Открытые (решить до начала этапа)

Вопрос	Варианты	Решить до
Судьба Unity-проекта	продолжать параллельно / заморозить как справочник / ParodiaR только полигон	этап 0
UI-библиотека	`bevy_ui` (встроен) или `bevy_egui` (удобнее для отладки, проверить версию под 0.20)	этап 1
Плавающее начало координат	сразу (`big_space`) или когда дрожь станет заметна (~10–100 км)	этап 5
Тени от солнца	нет / одна каскадная тень опцией	этап 10
Модели персонажей	кубоиды с пиксельными текстурами / воксельные части / гибрид	этап 18

Каждый закрытый вопрос — запись в `docs/DECISIONS.md`.

13 Словарь

Слово	Значение
Crate (пакет)	единица сборки Rust: библиотека или программа со своим `Cargo.toml`
Workspace	несколько пакетов в одной папке с общим `Cargo.lock` и `target`
ECS	Entity–Component–System: данные (компоненты) отдельно, логика (системы) отдельно
Plugin	кусок приложения Bevy, который сам регистрирует свои системы и ресурсы
SystemSet	именованная группа систем, чтобы задать порядок «эти — после тех»
FixedUpdate	расписание Bevy с постоянным шагом времени (физика, симуляция)
Message	сообщение «что-то произошло», которое читают другие системы
Чанк	кусок мира N×N×N блоков: единица загрузки, меша и сохранения
Меш	набор треугольников, который рисует видеокарта
Грязный чанк (dirty)	чанк, в котором что-то поменялось и меш надо перестроить
Детерминизм	одни и те же входы — всегда один и тот же результат
ADR	запись архитектурного решения: что решили, почему, какие были варианты

Слово	Значение
YAGNI	«не понадобится»: не писать код на будущее
DoD (Definition of Done)	проверяемое условие «этап готов»
LOD	уровень детализации: вдали рисуем грубее
SDF	функция расстояния до формы — способ описать пещеры и хребты формулой